Power-Optimized IEEE-754 Compliant Floating-Point Adder Using Dual-Path Architecture and Explicit Operand Isolation with Static Timing Analysis Validation

Ashit Raj*, Avni Jain*, Tanisha Gupta*, and Vishal Gupta*
*School of Electronics Engineering, Vellore Institute of Technology, Vellore, 632014, Tamil Nadu, India
Email: {ashit.raj2023, avni.jain2023, tanisha.gupta2023}@vitstudent.ac.in, vishal.gupta@vit.ac.in

***Abstract*—Floating-point arithmetic units are a dominant source of dynamic power dissipation in modern SoC designs, driven by high switching activity in their mantissa alignment and normalization stages. This paper presents a power and area optimized IEEE-754 Single-Precision floating-point adder implemented in Verilog and validated through post-synthesis physical performance analysis (PPA) using Cadence Genus. The design employs a *Dual-Path* micro-architecture that partitions the data path based on exponent difference (Δe), combined with RTL-level explicit operand isolation, special-case bypass, and a parallel priority-encoded Leading Zero Detector (LZD). Post-synthesis results from a 1,000-vector randomized testbench demonstrate a 10.9% reduction in dynamic switching power, a 15% reduction in silicon area, and 17% fewer standard cells over a monolithic baseline, with full IEEE-754 special-case compliance and timing closure at 200 MHz with positive setup slack, incurring only a 3.6% timing penalty.**

***Index Terms*—IEEE-754, floating-point adder, dual-path architecture, operand isolation, dynamic power reduction, leading zero detector, static timing analysis, area reduction, Cadence Genus, RTL synthesis, VLSI low-power design.**

I. Introduction

Floating-point (FP) arithmetic, standardized by the IEEE-754 specification [1], is fundamental to a broad spectrum of compute-intensive applications including scientific simulation, digital signal processing (DSP), graphics processing, and the inference engines of deep neural networks running on AI accelerators. Unlike fixed-point or integer arithmetic, floating-point addition imposes a substantially heavier hardware burden: before any arithmetic can proceed, both operands must be aligned to a common exponent via a variable-length shift, and the post-addition result must be re-normalized, operations that generate large amounts of switching activity even when the numerical result is trivial.

In CMOS technology, dynamic power dissipation is governed by $P_{\text{dyn}} = \alpha C_L V_{DD}^2 f$, where activity factor (the average number of transitions per clock cycle) is represented as α, C_L is the load capacitance, V_{DD} is the supply voltage, and f is the operating frequency. For a monolithic floating-point adder, the alignment barrel-shifter and normalization logic together account for a disproportionate share of total logic-level switching activity, since both blocks exercise long combinational paths on every single arithmetic operation, regardless of the actual magnitude difference between operands. Consequently, brute-force floating-point units become energetically inefficient for portable and thermally constrained SoC deployments.

A well-established strategy to mitigate this inefficiency is the *dual-path* (or *two-path*) architecture [2], which exploits the observation that the normalization requirement depends fundamentally on the exponent difference $\Delta e = |e_A - e_B|$ [3]. Further progress demonstrated low-latency dual-path designs [5], [9]. Fused dual-path units for DSP butterflies [6], [8] and energy-efficient FP multiply-add units [7] extended the approach to combined operations. Recently an implementation was made for a Far-and-Close path adder using Cadence Genus, prioritizing latency reduction [10], while approximate mantissa techniques have traded exactness for power savings [11]. Leading zero counter optimization for FPGA targets has been explored separately [12]. Complementary to architectural partitioning, *operand isolation* forces the inputs of inactive functional units to a constant, suppressing all internal transitions. Static Timing Analysis (STA) [4] using a DFF-wrapper methodology provides rigorous sign-off for the combinational datapath.

II. IEEE-754 Background

An IEEE-754 single precision floating-point number is a 32-bit quantity partitioned as shown in Fig. 1: one sign bit s, an 8-bit biased exponent e (bias $= 127$), and a 23-bit fractional mantissa f. The represented value is $(-1)^s \times 1.f \times 2^{e-127}$ for normal numbers ($1 \leq e \leq 254$). Special encodings include: *Zero* ($e = 0$, $f = 0$), *Denormal* ($e = 0$, $f \neq 0$), *Infinity* ($e = 255$, $f = 0$), and *NaN* ($e = 255$, $f \neq 0$).

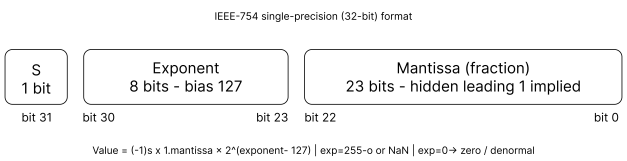

Fig. 1: IEEE-754 single precision format.

Adding two IEEE-754 numbers requires:

1) Exponent comparison to find Δe.
2) Mantissa alignment via right-shift, generating GRS (Guard, Round, and Sticky) bits.
3) Addition/Subtraction.
4) Leading zero detection.
5) Normalization.
6) Round-to-Nearest-Even rounding.

In a monolithic design, stages (2) and (5) each require a full 24-bit barrel-shifter active on every operation, constituting the core inefficiency addressed here. Dynamic power follows:

$$P_{\text{dyn}} = \alpha \cdot C_L \cdot V_{DD}^2 \cdot f \tag{1}$$

For a barrel-shifter with randomly varying shift amounts, $\alpha \approx 0.5$. Operand isolation clamps idle-path inputs to zero, driving $\alpha \to 0$ for that sub-datapath.

III. Proposed Architecture

The proposed `fp_dual_path_adder` addresses the core inefficiency of monolithic floating-point adders by replacing the single unified pipeline with two lightweight, specialized sub-datapaths. Rather than routing every operation through both a full alignment barrel-shifter and a full normalization barrel-shifter, the design inspects the operands immediately after exponent comparison and activates only the sub-datapath suited to that operand class. The top-level block diagram is shown in Fig. 2, and the complete operation flow in Fig. 3.

A. Special-Case Bypass

Before any arithmetic begins, both operands are unpacked into their constituent fields (sign, exponent, mantissa) and the hidden leading-1 bit is restored for normal numbers. An `early_bypass` flag (B_{early}) is then evaluated:

```
wire early_bypass = a_is_zero | b_is_zero
                  | (expA == 8'd255) | (expB == 8'd255);

wire [31:0] bypass_result =
    (expA == 8'd255) ? A :
    (expB == 8'd255) ? B :
    a_is_zero        ? B : A;
```

Listing 1: Special-case detection and bypass logic.

If either operand is zero, infinity, or NaN, B_{early} is asserted and the canonical IEEE-754 result is forwarded directly to the output, bypassing both arithmetic sub-datapaths entirely. This eliminates unnecessary computation and switching activity for all special-case inputs.

B. Path Routing

For normal operands, the exponent difference $\Delta e = |e_A - e_B|$ and the effective-subtraction flag $S_{eff} = s_A \oplus s_B$ are computed. These two signals jointly determine which sub-datapath is activated:

$$P_{close} = S_{eff} \cdot (\Delta e \leq 1) \cdot \overline{B_{early}} \tag{2}$$

When $P_{close} = 1$, the CLOSE path is selected; otherwise the FAR path is used. This Boolean decision is evaluated in a single logic level and introduces no additional pipeline stages.

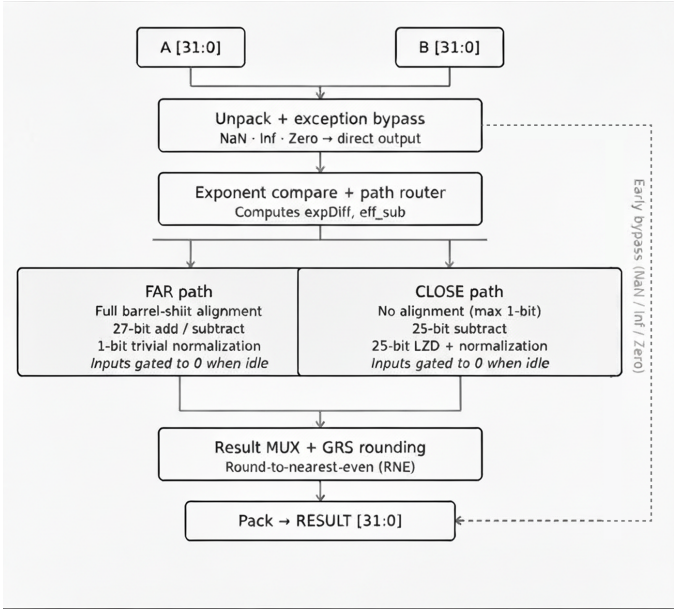

Fig. 2: Top-level block diagram of the proposed dual-path FP adder. Operand isolation gates are shown at each path input.

C. FAR Path ($\Delta e > 1$)

When the exponent difference exceeds one, the two operands differ by more than a factor of two in magnitude. In this regime, catastrophic cancellation — the phenomenon where subtraction of nearly equal values destroys all significant bits — is mathematically impossible. Consequently:

- The smaller mantissa is right-shifted by $\min(\Delta e, 26)$ positions using a 50-bit extended alignment shifter, preserving the guard (G), round (R), and sticky (S) bits in the low-order positions.
- Addition or subtraction is performed on 27-bit extended mantissa words.
- Because the result magnitude is guaranteed to be close to that of the larger operand, at most one bit of normalization adjustment is ever needed: either a single left-shift if the leading bit is zero after subtraction, or an exponent increment on overflow. The entire 24-bit normalization barrel-shifter present in the monolithic baseline is therefore eliminated.

D. CLOSE Path ($\Delta e \leq 1$, Effective Subtraction)

When two operands of opposite sign and nearly equal magnitude are subtracted, up to 24 leading zeros can appear in the result — a condition known as catastrophic cancellation. The CLOSE path is purpose-built for this case:

- Because $\Delta e \leq 1$, mantissa alignment requires at most a single-bit right-shift, so the alignment barrel-shifter is eliminated entirely.
- A 25-bit parallel priority-encoder Leading Zero Detector (LZD), implemented as a cascaded `if-else` chain that synthesizes to a balanced logic tree, counts the leading zeros in the subtraction result in a single pass.

- The result is left-shifted by the detected zero count and the exponent is decremented accordingly, completing normalization.

E. Operand Isolation

Even after path routing, the raw mantissa signals would propagate into the idle sub-datapath and cause unnecessary gate switching. To prevent this, RTL-level data barriers clamp the inputs of each idle path to zero:

```
// FAR path: frozen to zero when CLOSE is active
wire [23:0] far_mantBig   = is_far_path ? mantBig   : 24'd0
    ;
wire [23:0] far_mantSmall = is_far_path ? mantSmall : 24'd0
    ;

// CLOSE path: frozen to zero when FAR is active
wire [23:0] close_mantBig   = is_close_path ? mantBig   :
    24'd0;
wire [23:0] close_mantSmall = is_close_path ? mantSmall :
    24'd0;
```

Listing 2: Operand isolation for FAR and CLOSE paths.

With inputs held at zero, the downstream gates in the idle path receive no transitions. Their activity factor α is driven to zero, directly eliminating dynamic power dissipation from those blocks per Eq. (1).

F. Shared Rounding and Output Stage

The active path drives a common rounding stage that implements Round-to-Nearest-Even (RNE) using the three low-order bits preserved throughout computation:

$$R_{\text{up}} = G \cdot (R + S + \text{LSB}) \tag{3}$$

where G is the guard bit, R the round bit, S the sticky bit, and LSB the least-significant bit of the normalized mantissa. When $R_{\text{up}} = 1$, the mantissa is incremented by one and the exponent is corrected for any resulting overflow. If the post-rounding exponent underflows, the result is flushed to zero. The final sign, exponent, and mantissa fields are then packed into the 32-bit IEEE-754 output word.

IV. Methodology

Functional verification and activity profiling were performed using Cadence Xcelium / SimVision. A common testbench (`tb_fp_adder_core.v`) applied 1,000 randomized 32-bit input vectors using the `$random` system function, followed by four directed corner cases: $0+1$, $1+0$, $+\infty+1$, and $1+(-1)$. A VCD dump was generated at every hierarchy level:

```
$dumpfile("activity_profile.vcd");
$dumpvars(0, tb_fp_adder_core);
```

Listing 3: VCD instrumentation in testbench.

The resulting VCD file was consumed by Cadence Genus during power analysis, providing gate-level toggle rates derived from actual simulated switching activity rather than the tool's default statistical estimate of 20%.

A purely combinational block has no defined timing start or end points. To perform meaningful STA, a *structural wrapper* (`fp_adder_wrapper.v`) was constructed, placing 32-bit

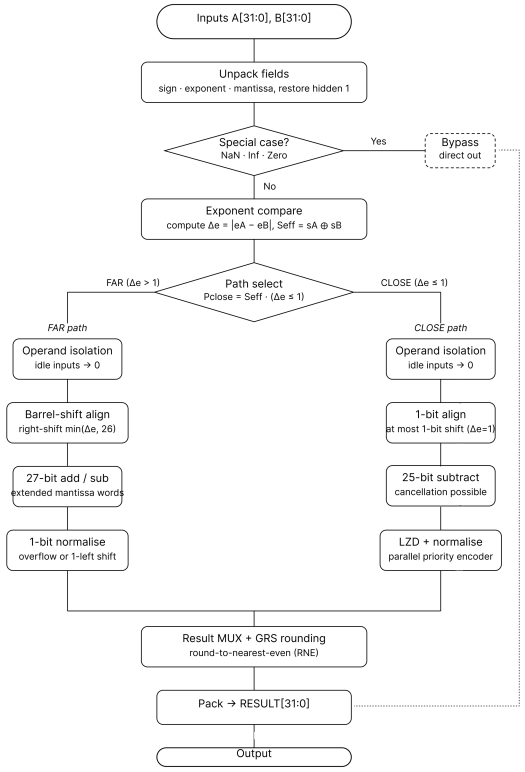

Fig. 3: Design flow of the proposed architecture.

DFF register banks at the A and B inputs and a 32-bit DFF register bank at the RESULT output. This creates a register-to-register (Reg2Reg) timing path and models the adder as it would appear inside a real CPU or DSP pipeline.

The following Synopsys Design Constraints (SDC) were applied identically to both designs:

- **Clock:** 200 MHz virtual clock with a 5.0 ns period.
- **Input/Output delays:** 0.5 ns at all register boundaries, leaving a 4.5 ns window for the combinational core.
- **Transition time:** 0.2 (normalized), preventing the tool from inserting unrealistically fast, power-hungry cells.
- **Load capacitance:** 80 (library units), ensuring realistic output drive strength assumptions.

These constraints represent industry-realistic sign-off conditions and were kept strictly identical to ensure a fair comparison.

Both designs were synthesized using Cadence Genus 21.14-s082_1 targeting the `PVT_1P1V_0C (balanced_tree)` standard-cell library corner. The synthesis objective was set to area-power balanced optimization (`balanced_tree` effort). All synthesis scripts, SDC files, and VCD activity profiles were kept identical between the two runs, differing only in the top-level RTL module under synthesis.

V. Results

No functional mismatches were observed across all 1,004 simulation vectors for either design. Path routing

TABLE I: Comparison With Prior Work

Metric	Galal (2011)	Sohn (2012)	Deka (2021)	Omidi (2021)	This Work
Technology	90 nm	45 nm	Std-cell	15 nm	45 nm
Tool	Custom	Std-cell	Genus + Innovus	Synopsys DC	Cadence Genus
Method	FMA, shared logic	Fused dual-path	Far-Close, Kogge-Stone	Approximate mantissa	Dual-path + isolation
Focus	Power density	Area / Power	Delay	Power (approx.)	Power / Area / STA
Delay / Performance	—	30% faster	16% faster than FCA	62% faster	3.6% slower (STA @ 200 MHz)
Power	1 W/mm^2	40% reduction	145 µW	37% reduction	**10.9% reduction**
Area	—	40% reduction	2190 cells	—	**15% reduction (894 cells)**
IEEE-754 Compliance	Yes	Yes	Yes	No	**Yes**

(`is_close_path`, `is_far_path`) alternated correctly with operand values, and the `early_bypass` path correctly forwarded results for special-case inputs.

Table I places the proposed design in context against prior work. The most directly comparable work is Deka et al. [10], which also employs the Far-and-Close algorithm and Cadence Genus. They report a delay of 6,347 ps (16% faster than FCA), at the cost of 2,190 cells and 145 µW dynamic power.

Their FCA baseline (1,248 cells, 113 µW) is comparable to our monolithic baseline (1,077 cells, 95 µW switching), indicating that the dual-path architecture alone does not inherently reduce power without additional techniques such as operand isolation.

In contrast, our proposed design achieves simultaneous reductions in switching power and area, while meeting timing closure and maintaining full IEEE-754 compliance.

The post-synthesis power breakdown is presented in Table II. The 10.9% reduction in switching power directly reflects the suppression of switching activity through operand isolation. The 4.2% increase in internal power arises from the additional capacitance introduced by dual-path routing multiplexers. Table III summarizes the post-synthesis area results. The

TABLE II: Post-Synthesis Power Comparison

Power Metric	Baseline	Proposed Architecture	Change
Leakage Power	1.063 µW	0.941 µW	−11.5%
Internal Power	233.08 µW	242.84 µW	+4.2%
Switching Power	95.02 µW	84.65 µW	**−10.9%**
Total	**329.16 µW**	**328.43 µW**	−0.2%

15% area reduction despite two sub-data paths plus a result mux demonstrates that architectural specialization outperforms monolithic generality. The baseline's dual barrel-shifters and iterative normalization loop occupy far more silicon.

TABLE III: Post-Synthesis Area Comparison

Area Metric	Baseline	Proposed Architecture	Change
Cell Count	1,077	894	**−17.0%**
Cell Area	1978.8 μm^2	1682.3 μm^2	**−15.0%**
Net Area	0.0 μm^2	0.0 μm^2	—
Total	**1978.8 μm^2**	**1682.3 μm^2**	−15.0%

The STA results for the baseline and our architecture are presented in Table IV. The proposed design is 118 ps (3.6%) slower than the baseline, attributable to the final 2-to-1 result multiplexer added to the critical path. Nevertheless, both designs report a large positive setup slack (greater than 1.5 ns), confirming that the proposed architecture is safe for 200 MHz operation with substantial timing margin, sufficient to absorb process variation in a real tape-out flow.

TABLE IV: Static Timing Analysis Results

Timing Metric	Baseline	Proposed Architecture	Diff.
Data Path Delay	3218 ps	3336 ps	$+118\,ps$
Required Time	5387 ps	5386 ps	—
Setup Slack	$+1669\,ps$	$+1550\,ps$	$-119\,ps$
Met?	**YES**	**YES**	—

Table V consolidates all post-synthesis PPA metrics for both designs, highlighting the simultaneous improvements achieved by the proposed architecture across power, area, and timing dimensions.

TABLE V: Consolidated PPA Summary

Metric	Baseline	Proposed Architecture	Impr.
Switching Power	95.02 μW	84.65 μW	−10.9%
Leakage Power	1.063 μW	0.941 μW	−11.5%
Total Power	329.16 μW	328.43 μW	−0.2%
Cell Count	1,077	894	−17.0%
Cell Area	1978.8 μm^2	1682.3 μm^2	−15.0%
Data Path Delay	3218 ps	3336 ps	+3.6%
Setup Slack (5 ns)	+1669 ps	+1550 ps	Passed

The proposed architecture achieves simultaneous improvement in switching power and area without timing degradation beyond acceptable margins.

VI. Conclusion

This paper presented a power optimized IEEE-754 compliant single precision floating-point adder using three techniques: dual-path architectural partitioning, RTL-level explicit operand isolation, and a parallel priority-encoded Leading Zero Detector. Post-synthesis analysis with Cadence Genus, driven by VCD

activity profiles from a 1,004-vector simulation under Cadence Xcelium, demonstrated a 10.9% reduction in dynamic switching power, a 15% reduction in silicon area, full IEEE-754 special-case compliance, and timing closure at 200 MHz with $+1550$ ps positive setup slack.

These results demonstrate that power-aware architectural partitioning — targeting the specific stages responsible for excess switching activity — yields measurable improvements that synthesis-tool directives alone cannot achieve. The proposed architecture eliminates logic that is structurally unnecessary for a given operand class rather than merely optimizing logic that is there, fundamentally reducing both area and power.

Future work includes extension to double-precision (64-bit) operands, exploration of approximate computing for the CLOSE path in low-criticality inference workloads, and physical implementation in a sub-28 nm library to evaluate routing-congestion and parasitic effects on the reported improvements.

REFERENCES

[1] IEEE Standards Association, "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2008*, Aug. 2008. DOI: 10.1109/IEEESTD.2008.4610935.

[2] J. D. Bruguera and T. Lang, "Leading-One Prediction with Concurrent Position Correction," *IEEE Trans. Comput.*, vol. 48, no. 10, pp. 1083–1097, Oct. 1999. DOI: 10.1109/12.805157.

[3] S. F. Oberman and M. J. Flynn, "Design Issues in Division and Other Floating-Point Operations," *IEEE Trans. Comput.*, vol. 46, no. 2, pp. 154–161, Feb. 1997. DOI: 10.1109/12.565590.

[4] J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*, Springer, 2009. ISBN: 978-0-387-93819-6.

[5] P.-M. Seidel and G. Even, "Delay-Optimized Implementation of IEEE Floating-Point Addition," *IEEE Trans. Comput.*, vol. 53, no. 2, pp. 97–113, Feb. 2004. DOI: 10.1109/TC.2004.1261822.

[6] J. Sohn and E. E. Swartzlander, "Improved Architectures for a Fused Floating-Point Add-Subtract Unit," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 59, no. 10, pp. 2285–2291, Oct. 2012. DOI: 10.1109/TCSI.2012.2188946.

[7] S. Galal and M. Horowitz, "Energy-Efficient Floating-Point Unit Design," *IEEE Trans. Comput.*, vol. 60, no. 7, pp. 913–922, Jul. 2011. DOI: 10.1109/TC.2010.121.

[8] E. E. Swartzlander and H. H. Saleh, "FFT Implementation with Fused Floating-Point Operations," *IEEE Trans. Comput.*, vol. 61, no. 2, pp. 284–288, Feb. 2012. DOI: 10.1109/TC.2010.271.

[9] A. Beaumont-Smith, N. Burgess, S. Lefrere, and C.-C. Lim, "Reduced Latency IEEE Floating-Point Standard Adder Architectures," in *Proc. 14th IEEE Symp. Comput. Arithmetic (ARITH)*, Adelaide, Australia, Apr. 1999, pp. 35–42. DOI: 10.1109/ARITH.1999.762831.

[10] D. Deka, N. Kumar, and D. Pal, "Design of an ASIC-Based High Speed 32-bit Floating Point Adder," in *Proc. 2021 Int. Conf. Applied Electronics (AE)*, Pilsen, Czech Republic, Sep. 2021. DOI: 10.23919/AE51540.2021.9542881.

[11] R. Omidi and S. Sharifzadeh, "Design of Low Power Approximate Floating-Point Adders," *Int. J. Circuit Theory Appl.*, vol. 49, no. 1, pp. 185–195, Jan. 2021. DOI: 10.1002/cta.2831.

[12] S. Perri, F. Spagnolo, F. Frustaci, and P. Corsonello, "Design of Leading Zero Counters on FPGAs," *IEEE Embedded Syst. Lett.*, vol. 15, no. 3, pp. 149–152, Sep. 2023. DOI: 10.1109/LES.2022.3217861.